=== mobile/src/components/AnimatedCheckbox.tsx ===

```tsx
import React from 'react';
import { Pressable, View, StyleSheet } from 'react-native';
import Svg, { Path } from 'react-native-svg';

/**
 * 动画复选框组件属性
 */
interface CheckboxProps {
  /** 是否选中 */
  checked: boolean;
  /** 切换选中状态的回调 */
  onToggle: () => void;
  /** 复选框大小 */
  size?: number;
  /** 复选框颜色 */
  color?: string;
}

/**
 * 动画复选框组件
 * 提供自定义样式的复选框，包含SVG对勾
 */
const AnimatedCheckbox: React.FC<CheckboxProps> = ({
  checked,
  onToggle,
  size = 24,
  color = '#80FF00',
}) => {
  return (
    <Pressable onPress={onToggle} style={styles.container}>
      <View style={[styles.box, { width: size * 1.5, height: size * 1.5, borderColor: color }]}>
        {/* 选中时显示SVG对勾 */}
        {checked && (
          <Svg
            width={size * 1.5}
            height={size * 1.5}
            viewBox="0 0 72 97"
            style={styles.checkmark}
          >
            <Path
              d="M28.72 95.673c-6.37.644-11.034-4.445-15.746-8.048-4.947-3.783-8.859-10.482-10.847-16.446-6.68
              strokeWidth="4px"
              stroke={color}
              fill="none"
              strokeLinecap="round"
              strokeLinejoin="round"
            />
          </Svg>
        )}
```

```
      </View>
    </Pressable>
  );
};

const styles = StyleSheet.create({
  /** 容器样式 */
  container: {
    justifyContent: 'center',
    alignItems: 'center',
  },
  /** 复选框边框样式 */
  box: {
    borderWidth: 2,
    borderRadius: 4,
    justifyContent: 'center',
    alignItems: 'center',
    overflow: 'hidden',
  },
  /** 对勾样式 */
  checkmark: {
    position: 'absolute',
  },
});

export default AnimatedCheckbox;

=== mobile/src/components/FloatingInputBar.tsx ===
import React, { useRef, useEffect } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  Animated,
  Keyboard,
  Platform,
  KeyboardEvent,
  StyleSheet,
  InteractionManager,
  KeyboardTypeOptions,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';

// ============================================================
// FloatingInputBar
// 键盘上方浮动输入栏（极简版）
//
// 点击页面上的"伪输入框"后，浮层栏出现在键盘上方并自动聚焦。
// 输入内容实时同步到原始字段。点「完成」收起键盘。
```

```
// ================================================================

export interface InputFieldConfig {
  id: string;
  label: string;
  icon: string;
  value: string;
  onChangeText: (text: string) => void;
  keyboardType?: KeyboardTypeOptions;
  secureTextEntry?: boolean;
  placeholder?: string;
}

interface FloatingInputBarProps {
  fields: InputFieldConfig[];
  activeFieldId: string | null;
  onDismiss: () => void;
  colors: {
    backgroundDark: string;
    background: string;
    primary: string;
    textPrimary: string;
    textSecondary: string;
    borderLight: string;
    border: string;
  };
}

const BAR_BOTTOM_SPACING = 8; // 输入栏与键盘之间的间距

export const FloatingInputBar: React.FC<FloatingInputBarProps> = ({
  fields,
  activeFieldId,
  onDismiss,
  colors,
}) => {
  const inputRef = useRef<TextInput>(null);
  const translateY = useRef(new Animated.Value(0)).current;
  const opacity = useRef(new Animated.Value(0)).current;

  const activeField = fields.find((f) => f.id === activeFieldId);

  // ---- 字段切换或首次挂载时聚焦 ----
  // key={activeFieldId} 确保每次切换都创建新的 TextInput 实例
  // autoFocus 处理首次挂载，InteractionManager 确保动画完成后聚焦
  useEffect(() => {
    if (activeFieldId) {
      InteractionManager.runAfterInteractions(() => {
        if (inputRef.current) {
          inputRef.current.focus();
```

```
      }
    });
  }
}, [activeFieldId]);

// ---- 键盘动画 ----
useEffect(() => {
  const showEvent =
    Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow';
  const hideEvent =
    Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide';

  const showSub = Keyboard.addListener(showEvent, (e: KeyboardEvent) => {
    // translateY 为负 = 向上移动，减去 BAR_BOTTOM_SPACING 留出间距
    const targetY = -(e.endCoordinates.height + BAR_BOTTOM_SPACING);

    Animated.parallel([
      Animated.timing(translateY, {
        toValue: targetY,
        duration: e.duration || 280,
        useNativeDriver: true,
      }),
      Animated.timing(opacity, {
        toValue: 1,
        duration: (e.duration || 280) * 0.5,
        useNativeDriver: true,
      }),
    ]).start();
  });

  const hideSub = Keyboard.addListener(hideEvent, (e: KeyboardEvent) => {
    Animated.parallel([
      Animated.timing(translateY, {
        toValue: 0,
        duration: e.duration || 280,
        useNativeDriver: true,
      }),
      Animated.timing(opacity, {
        toValue: 0,
        duration: (e.duration || 280) * 0.6,
        useNativeDriver: true,
      }),
    ]).start();
  });

  return () => {
    showSub.remove();
    hideSub.remove();
  };
}, [translateY, opacity]);
```

```
199
200 // ---- 收起 ----
201 const handleDone = () => {
202   onDismiss();
203 };
204
205 if (!activeField) return null;
206
207 return (
208   <Animated.View
209     style={[
210       styles.container,
211       {
212         backgroundColor: colors.backgroundDark,
213         borderTopColor: colors.borderLight,
214       },
215       {
216         transform: [{ translateY }],
217         opacity,
218       },
219     ]}
220   >
221     {/* 输入行 */}
222     <View
223       style={[
224         styles.inputRow,
225         {
226           backgroundColor: colors.background,
227           borderColor: colors.borderLight,
228         },
229       ]}
230     >
231       <Ionicons
232         name={activeField.icon as any}
233         size={18}
234         color={colors.primary}
235         style={styles.inputIcon}
236       />
237       <TextInput
238         key={activeFieldId}
239         ref={inputRef}
240         style={[styles.textInput, { color: colors.textPrimary }]}
241         value={activeField.value}
242         onChangeText={activeField.onChangeText}
243         placeholder={activeField.placeholder}
244         placeholderTextColor={colors.textSecondary}
245         keyboardType={activeField.keyboardType || 'default'}
246         secureTextEntry={activeField.secureTextEntry}
247         autoCapitalize="none"
248         autoCorrect={false}
```

```
        autoFocus
        returnKeyType="done"
        onSubmitEditing={handleDone}
      />
      {activeField.value.length > 0 && (
        <TouchableOpacity
          onPress={() => activeField.onChangeText('')}
          style={styles.clearButton}
          hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
        >
          <Ionicons name="close-circle" size={18} color={colors.textSecondary} />
        </TouchableOpacity>
      )}
      <View style={styles.doneDivider} />
      <TouchableOpacity onPress={handleDone} style={styles.doneButton}>
        <Text style={[styles.doneText, { color: colors.primary }]}>完成</Text>
      </TouchableOpacity>
    </View>
  </Animated.View>
 );
};

const styles = StyleSheet.create({
  container: {
    position: 'absolute',
    bottom: 0,
    left: 0,
    right: 0,
    borderTopWidth: StyleSheet.hairlineWidth,
    paddingVertical: 10,
    paddingHorizontal: 12,
  },
  inputRow: {
    flexDirection: 'row',
    alignItems: 'center',
    borderRadius: 14,
    borderWidth: 1,
    paddingHorizontal: 14,
    height: 48,
  },
  inputIcon: {
    marginRight: 10,
  },
  textInput: {
    flex: 1,
    fontSize: 16,
    paddingVertical: 0,
  },
  clearButton: {
    padding: 4,
```

```
    marginLeft: 4,
  },
  doneDivider: {
    width: 1,
    height: 20,
    backgroundColor: 'rgba(128,128,128,0.25)',
    marginHorizontal: 10,
  },
  doneButton: {
    paddingVertical: 6,
    paddingHorizontal: 4,
    borderRadius: 8,
  },
  doneText: {
    fontSize: 15,
    fontWeight: '700',
  },
});

=== mobile/src/components/GameHelpModal.tsx ===
import React, { useMemo } from 'react';
import { View, Text, StyleSheet, Modal, TouchableOpacity, ScrollView } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useTheme } from '../contexts/ThemeContext';

interface GameHelpModalProps {
  visible: boolean;
  onClose: () => void;
}

const GameHelpModal: React.FC<GameHelpModalProps> = ({ visible, onClose }) => {
  const { colors } = useTheme();

  const styles = useMemo(() => StyleSheet.create({
    container: {
      flex: 1,
      backgroundColor: colors.background,
    },
    header: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
      padding: 16,
      paddingTop: 48,
      borderBottomWidth: 1,
      borderBottomColor: colors.borderDark,
    },
    closeBtn: { padding: 8 },
    title: {
      color: colors.textPrimary,
```

```
    fontSize: 20,
    fontWeight: '700',
  },
  placeholder: { width: 44 },
  content: { padding: 16 },
  section: {
    marginBottom: 24,
    backgroundColor: colors.borderLight,
    borderRadius: 12,
    padding: 16,
    borderWidth: 1,
    borderColor: colors.border,
  },
  sectionTitle: {
    color: colors.warning,
    fontSize: 16,
    fontWeight: '700',
    marginBottom: 8,
  },
  sectionDesc: {
    color: colors.textSecondary,
    fontSize: 14,
    lineHeight: 22,
  },
  rulesList: { gap: 6 },
  ruleItem: {
    color: colors.textSecondary,
    fontSize: 14,
    lineHeight: 22,
  },
}), [colors]);

return (
  <Modal
    visible={visible}
    animationType="slide"
    onRequestClose={onClose}
  >
    <View style={styles.container}>
      <View style={styles.header}>
        <TouchableOpacity onPress={onClose} style={styles.closeBtn}>
          <Ionicons name="close" size={28} color={colors.textSecondary} />
        </TouchableOpacity>
        <Text style={styles.title}>游戏说明</Text>
        <View style={styles.placeholder} />
      </View>

      <ScrollView style={styles.content} showsVerticalScrollIndicator={false}>
        <View style={styles.section}>
          <Text style={styles.sectionTitle}> 游戏恢复体力</Text>
```

```
          <Text style={styles.sectionDesc}>
            每天可游玩3次小游戏恢复体力，完成游戏后可获得体力和能量奖励。
          </Text>
        </View>

        <View style={styles.section}>
          <Text style={styles.sectionTitle}> 数独</Text>
          <Text style={styles.sectionDesc}>
            在9×9的网格中填入数字1-9，使每行、每列、每个3×3宫格内的数字都不重复。完成一局即可获得奖励！
          </Text>
        </View>

        <View style={styles.section}>
          <Text style={styles.sectionTitle}> 推箱子</Text>
          <Text style={styles.sectionDesc}>
            将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
          </Text>
        </View>

        <View style={styles.section}>
          <Text style={styles.sectionTitle}> 游戏规则</Text>
          <View style={styles.rulesList}>
            <Text style={styles.ruleItem}>• 数独：完成一局即可获得奖励</Text>
            <Text style={styles.ruleItem}>• 推箱子：完成3关即可获得奖励</Text>
            <Text style={styles.ruleItem}>• 每天可游玩3次，凌晨5点重置次数</Text>
            <Text style={styles.ruleItem}>• 完成游戏后奖励会自动添加到你的账户</Text>
            <Text style={styles.ruleItem}>• 游戏过程中可随时退出，不会消耗次数</Text>
          </View>
        </View>
      </ScrollView>
    </View>
  </Modal>
 );
};

export default GameHelpModal;

=== mobile/src/components/GameModal.tsx ===
import React, { useState, useEffect, useMemo } from 'react';
import { View, Text, StyleSheet, TouchableOpacity, ScrollView, Alert, Modal } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useTheme } from '../contexts/ThemeContext';
import {
 sokobanLevels,
 getLevelByIndex,
 calculateRewards,
 mockSubmitGameResult,
 type GameResult,
} from '../services/gameService';
```

```
// ==================== 类型定义 ====================

/**
 * 游戏弹窗组件属性
 */
interface GameModalProps {
  /** 是否显示弹窗 */
  visible: boolean;
  /** 关闭弹窗回调 */
  onClose: () => void;
  /** 游戏完成回调，返回奖励 */
  onGameComplete: (rewards: { stamina: number; energy: number }) => void;
  /** 怪兽类型，影响奖励计算 */
  monsterType?: string;
}

/**
 * 当前显示屏幕类型
 */
type Screen = 'welcome' | 'tutorial' | 'game';

/**
 * 游戏类型
 */
type GameType = 'sudoku' | 'sokoban';

// ==================== 数独游戏组件 ====================

/**
 * 数独游戏组件
 */
const SudokuGame = ({ onWin, colors }: { onWin: () => void; colors: any }) => {
  /** 游戏面板，0表示空 */
  const [board, setBoard] = useState<number[][]>([]);
  /** 答案面板 */
  const [solution, setSolution] = useState<number[][]>([]);
  /** 哪些位置是初始给定的（不可修改） */
  const [given, setGiven] = useState<boolean[][]>([]);
  /** 当前选中的格子位置 */
  const [selected, setSelected] = useState<{ row: number; col: number } | null>(null);

  /**
   * 检查数字在当前位置是否有效
   */
  const isValid = (grid: number[][], row: number, col: number, num: number): boolean => {
    // 检查行
    for (let i = 0; i < 9; i++) {
      if (grid[row][i] === num || grid[i][col] === num) return false;
    }
    // 检查3x3宫格
```

```
  const boxRow = Math.floor(row / 3) * 3;
  const boxCol = Math.floor(col / 3) * 3;
  for (let i = 0; i < 3; i++) {
    for (let j = 0; j < 3; j++) {
      if (grid[boxRow + i][boxCol + j] === num) return false;
    }
  }
  return true;
};

/**
 * 使用回溯法生成数独答案
 */
const solveSudoku = (grid: number[][]): boolean => {
  for (let row = 0; row < 9; row++) {
    for (let col = 0; col < 9; col++) {
      if (grid[row][col] === 0) {
        // 随机打乱数字1-9
        const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9].sort(() => Math.random() - 0.5);
        for (const num of nums) {
          if (isValid(grid, row, col, num)) {
            grid[row][col] = num;
            if (solveSudoku(grid)) return true;
            grid[row][col] = 0;
          }
        }
        return false;
      }
    }
  }
  return true;
};

/**
 * 初始化游戏
 */
const initGame = () => {
  // 生成答案
  const newSolution = Array(9).fill(0).map(() => Array(9).fill(0));
  solveSudoku(newSolution);

  // 复制答案到面板，然后挖空一些格子
  const newBoard = newSolution.map(row => [...row]);
  const newGiven = Array(9).fill(0).map(() => Array(9).fill(false));

  // 随机挖空35个格子
  const cellsToRemove = 35;
  let removed = 0;
  while (removed < cellsToRemove) {
    const row = Math.floor(Math.random() * 9);
```

```
      const col = Math.floor(Math.random() * 9);
      if (newBoard[row][col] !== 0) {
        newBoard[row][col] = 0;
        removed++;
      }
    }

    // 标记哪些是给定的
    for (let row = 0; row < 9; row++) {
      for (let col = 0; col < 9; col++) {
        newGiven[row][col] = newSolution[row][col] === newBoard[row][col] && newBoard[row][col] !== 0;
      }
    }

    setSolution(newSolution);
    setBoard(newBoard);
    setGiven(newGiven);
    setSelected(null);
  };

  // 组件挂载时初始化游戏
  useEffect(() => {
    initGame();
  }, []);

  /**
   * 检查游戏是否胜利
   */
  const checkWin = (): boolean => {
    for (let row = 0; row < 9; row++) {
      for (let col = 0; col < 9; col++) {
        if (board[row][col] !== solution[row][col]) return false;
      }
    }
    return true;
  };

  /**
   * 输入数字处理
   */
  const handleInput = (num: number) => {
    if (!selected || given[selected.row][selected.col]) return;

    const newBoard = board.map(row => [...row]);
    newBoard[selected.row][selected.col] = num;
    setBoard(newBoard);

    // 检查胜利
    if (checkWin()) {
      setTimeout(() => onWin(), 300);
```

```
  }
};

/**
 * 清除数字
 */
const handleClear = () => {
  if (!selected || given[selected.row][selected.col]) return;

  const newBoard = board.map(row => [...row]);
  newBoard[selected.row][selected.col] = 0;
  setBoard(newBoard);
};

/**
 * 获取格子背景色
 */
const getCellColor = (row: number, col: number): string => {
  if (!selected) return colors.background;
  if (selected.row === row && selected.col === col) return colors.borderDark;
  if (selected.row === row || selected.col === col) return colors.borderLight;
  const boxRow = Math.floor(selected.row / 3) * 3;
  const boxCol = Math.floor(selected.col / 3) * 3;
  if (row >= boxRow && row < boxRow + 3 && col >= boxCol && col < boxCol + 3) {
    return colors.borderLight;
  }
  return colors.background;
};

return (
  <View style={styles.gameContainer}>
    {/* 头部 */}
    <View style={styles.sudokuHeader}>
      <Text style={styles.sudokuTitle}> 数独</Text>
      <TouchableOpacity style={styles.newGameBtn} onPress={initGame}>
        <Ionicons name="refresh" size={16} color={colors.primary} />
        <Text style={styles.newGameText}>新游戏</Text>
      </TouchableOpacity>
    </View>

    {/* 游戏面板 */}
    <View style={styles.sudokuGrid}>
      {board.map((row, rowIndex) => (
        <View key={rowIndex} style={styles.sudokuRow}>
          {row.map((cell, colIndex) => (
            <TouchableOpacity
              key={colIndex}
              style={[styles.sudokuCell, { backgroundColor: getCellColor(rowIndex, colIndex) }]}
              onPress={() => !given[rowIndex][colIndex] && setSelected({ row: rowIndex, col: colIndex })}
              disabled={given[rowIndex][colIndex]}
```

```
            >
              {cell !== 0 && (
                <Text style={[
                  styles.sudokuCellText,
                  given[rowIndex][colIndex] ? styles.sudokuGiven : styles.sudokuInput
                ]}>
                  {cell}
                </Text>
              )}
            </TouchableOpacity>
          ))}
        </View>
      ))}
    </View>

    {/* 数字键盘 */}
    <View style={styles.numberPad}>
      {[1, 2, 3, 4, 5, 6, 7, 8, 9].map(num => (
        <TouchableOpacity
          key={num}
          style={styles.numberBtn}
          onPress={() => handleInput(num)}
        >
          <Text style={styles.numberBtnText}>{num}</Text>
        </TouchableOpacity>
      ))}
      <TouchableOpacity style={styles.clearBtn} onPress={handleClear}>
        <Text style={styles.clearBtnText}>清除</Text>
      </TouchableOpacity>
    </View>

    </View>
  );
};

// ==================== 推箱子游戏组件 ====================

/**
 * 推箱子游戏组件
 */
const SokobanGame = ({ onWin, monsterType, colors }: { onWin: (level: number) => void; monsterType?: string; c
  /** 当前关卡索引 */
  const [currentLevel, setCurrentLevel] = useState(0);
  /** 玩家位置 */
  const [playerPos, setPlayerPos] = useState({ x: 0, y: 0 });
  /** 游戏地图 */
  const [gameMap, setGameMap] = useState<string[][]>([]);

  /**
   * 初始化指定关卡
```

```
 */
const initLevel = (levelIndex: number) => {
  const level = getLevelByIndex(levelIndex);
  const newMap = level.map.map(row => row.split(''));
  let pPos = { x: 0, y: 0 };

  // 找到玩家位置并将其从地图中移除
  for (let y = 0; y < newMap.length; y++) {
    for (let x = 0; x < newMap[y].length; x++) {
      if (newMap[y][x] === '@') {
        pPos = { x, y };
        newMap[y][x] = ' ';
      }
    }
  }

  setCurrentLevel(levelIndex);
  setGameMap(newMap);
  setPlayerPos(pPos);
};

// 组件挂载时初始化第一关
useEffect(() => {
  initLevel(0);
}, []);

/**
 * 检查当前地图是否胜利
 */
const checkWin = (map: string[][]): boolean => {
  for (const row of map) {
    if (row.includes('$') || row.includes('.')) return false;
  }
  return true;
};

/**
 * 移动玩家
 */
const move = (dx: number, dy: number) => {
  // 重置当前关卡
  if (dx === 0 && dy === 0) {
    initLevel(currentLevel);
    return;
  }

  const newX = playerPos.x + dx;
  const newY = playerPos.y + dy;

  // 检查边界
```

```
    if (newY < 0 || newY >= gameMap.length || newX < 0 || newX >= gameMap[newY].length) return;

    const target = gameMap[newY][newX];
    if (target === '#') return; // 撞墙

    const newMap = gameMap.map(row => [...row]);

    // 推箱子
    if (target === '$' || target === '*') {
      const boxNewX = newX + dx;
      const boxNewY = newY + dy;

      // 检查箱子移动位置
      if (boxNewY < 0 || boxNewY >= gameMap.length || boxNewX < 0 || boxNewX >= gameMap[boxNewY].length) retur

      const boxTarget = gameMap[boxNewY][boxNewX];
      if (boxTarget === ' ' || boxTarget === '.') {
        newMap[newY][newX] = target === '*' ? '.' : ' ';
        newMap[boxNewY][boxNewX] = boxTarget === '.' ? '*' : '$';
      } else {
        return;
      }
    }

    setGameMap(newMap);
    setPlayerPos({ x: newX, y: newY });

    // 检查胜利
    const currentLevelCopy = currentLevel;
    if (checkWin(newMap)) {
      setTimeout(() => {
        if (currentLevelCopy < 2) {
          initLevel(currentLevelCopy + 1);
        } else {
          onWin(currentLevelCopy + 1);
        }
      }, 500);
    }
  };

  /**
   * 获取格子内容和样式
   */
  const getCellContent = (char: string, x: number, y: number): { type: string; content: string } => {
    if (playerPos.x === x && playerPos.y === y) {
      return { type: 'player', content: '' };
    }
    switch (char) {
      case '#': return { type: 'wall', content: '' };
      case '$': return { type: 'box', content: '' };
```

```
    case '*': return { type: 'box-on-target', content: '' };
    case '.': return { type: 'target', content: '' };
    default: return { type: 'floor', content: '' };
  }
};

const level = getLevelByIndex(currentLevel);

return (
  <View style={styles.gameContainer}>
    {/* 头部 */}
    <View style={styles.sokobanHeader}>
      <Text style={styles.levelName}>{level.name} / {sokobanLevels.length}</Text>
      <TouchableOpacity style={styles.newGameBtn} onPress={() => initLevel(0)}>
        <Ionicons name="refresh" size={16} color={colors.primary} />
        <Text style={styles.newGameText}>重新开始</Text>
      </TouchableOpacity>
    </View>

    {/* 游戏地图 */}
    <View style={styles.sokobanGrid}>
      {gameMap.map((row, rowIndex) => (
        <View key={rowIndex} style={styles.sokobanRow}>
          {row.map((char, colIndex) => {
            const { type, content } = getCellContent(char, colIndex, rowIndex);
            const cellStyle = type === 'wall' ? styles.sokobanCellWall :
                              type === 'floor' ? styles.sokobanCellFloor :
                              type === 'target' ? styles.sokobanCellTarget :
                              type === 'player' ? styles.sokobanCellPlayer :
                              type === 'box' ? styles.sokobanCellBox :
                              styles.sokobanCellBoxOnTarget;
            return (
              <View key={colIndex} style={[styles.sokobanCell, cellStyle]}>
                <Text style={styles.sokobanCellText}>{content}</Text>
              </View>
            );
          })}
        </View>
      ))}
    </View>

    {/* 方向控制 */}
    <View style={styles.sokobanControls}>
      <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, -1)}>
        <Text style={styles.sokobanBtnText}></Text>
      </TouchableOpacity>
      <View style={styles.sokobanBtnRow}>
        <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(-1, 0)}>
          <Text style={styles.sokobanBtnText}></Text>
        </TouchableOpacity>
```

```
          <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 0)}>
            <Text style={styles.sokobanBtnText}></Text>
          </TouchableOpacity>
          <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(1, 0)}>
            <Text style={styles.sokobanBtnText}></Text>
          </TouchableOpacity>
        </View>
        <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 1)}>
          <Text style={styles.sokobanBtnText}></Text>
        </TouchableOpacity>
      </View>

      </View>
  );
};

// ==================== 主游戏弹窗组件 ====================

/**
 * 游戏弹窗主组件
 * 包含游戏选择、教程和游戏界面
 */
const GameModal = ({ visible, onClose, onGameComplete, monsterType }: GameModalProps) => {
  const { colors } = useTheme();

// ==================== 样式 ====================

const styles = useMemo(() => StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: colors.background,
  },
  header: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: 16,
    paddingTop: 40,
    borderBottomWidth: 1,
    borderBottomColor: 'colors.borderDark',
  },
  closeBtn: {
    padding: 8,
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    color: colors.textPrimary,
  },
  placeholder: {
```

```
    width: 44,
  },
  welcomeContent: {
    flex: 1,
    padding: 16,
  },
  welcomeSection: {
    textAlign: 'center',
    marginBottom: 24,
  },
  welcomeTitle: {
    fontSize: 28,
    fontWeight: 'bold',
    color: colors.primary,
    marginBottom: 8,
  },
  welcomeSubtitle: {
    fontSize: 14,
    color: colors.textSecondary,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: colors.warning,
    marginBottom: 12,
  },
  rewardsSection: {
    backgroundColor: 'rgba(255,215,0,0.08)',
    borderRadius: 12,
    padding: 16,
    marginBottom: 16,
  },
  rewardsRow: {
    flexDirection: 'row',
    justifyContent: 'center',
    gap: 40,
  },
  rewardBox: {
    alignItems: 'center',
  },
  rewardIcon: {
    fontSize: 32,
    marginBottom: 4,
  },
  rewardAmount: {
    fontSize: 20,
    fontWeight: 'bold',
    color: colors.warning,
  },
  rewardName: {
```

```
    fontSize: 12,
    color: colors.textSecondary,
  },
  rewardNote: {
    fontSize: 12,
    color: colors.textSecondary,
    textAlign: 'center',
    marginTop: 12,
  },
  gamesSection: {
    marginBottom: 16,
  },
  gameCard: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: colors.surface,
    borderRadius: 12,
    padding: 16,
    marginBottom: 12,
    borderWidth: 1,
    borderColor: 'colors.borderDark',
  },
  gameCardIcon: {
    fontSize: 36,
    marginRight: 12,
  },
  gameCardInfo: {
    flex: 1,
  },
  gameCardTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: colors.textPrimary,
    marginBottom: 4,
  },
  gameCardDesc: {
    fontSize: 12,
    color: colors.textSecondary,
  },
  gameCardArrow: {
    fontSize: 20,
    color: colors.primary,
  },
  tipsSection: {
    backgroundColor: 'rgba(123,117,216,0.08)',
    borderRadius: 12,
    padding: 16,
  },
  tipsText: {
    fontSize: 12,
```

```
    color: colors.textSecondary,
    lineHeight: 20,
  },
  tipsList: {
    gap: 6,
  },
  tipItem: {
    fontSize: 12,
    color: colors.textSecondary,
    lineHeight: 20,
  },
  tutorialContent: {
    flex: 1,
    padding: 16,
  },
  backBtn: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 4,
    marginBottom: 16,
    zIndex: 200,
    position: 'relative',
  },
  backBtnText: {
    color: colors.textSecondary,
    fontSize: 14,
  },
  tutorialHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    marginBottom: 20,
  },
  tutorialIcon: {
    fontSize: 40,
  },
  tutorialTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: colors.textPrimary,
  },
  tutorialSection: {
    marginBottom: 24,
    paddingBottom: 8,
  },
  tutorialSectionTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: colors.warning,
    marginBottom: 8,
```

```
  },
  tutorialText: {
    fontSize: 14,
    color: colors.textSecondary,
    lineHeight: 22,
  },
  tutorialSteps: {
    gap: 6,
  },
  tutorialStep: {
    fontSize: 14,
    color: colors.textSecondary,
    lineHeight: 22,
  },
  startBtn: {
    backgroundColor: colors.primary,
    borderRadius: 12,
    padding: 16,
    alignItems: 'center',
    marginTop: 32,
    marginBottom: 32,
  },
  startBtnText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#FFFFFF',
  },
  gameScreen: {
    flex: 1,
    padding: 16,
    paddingTop: 40,
  },
  gameContent: {
    flex: 1,
  },
  gameContainer: {
    flex: 1,
  },
  // 数独样式
  sudokuHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 16,
  },
  sudokuTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: colors.textPrimary,
  },
```

```
  newGameBtn: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 4,
    padding: 8,
    backgroundColor: 'colors.borderLight',
    borderRadius: 8,
  },
  newGameText: {
    fontSize: 12,
    color: colors.primary,
  },
  sudokuGrid: {
    backgroundColor: colors.backgroundDark,
    borderRadius: 8,
    padding: 4,
    marginBottom: 16,
    borderWidth: 2,
    borderColor: colors.primary,
  },
  sudokuRow: {
    flexDirection: 'row',
  },
  sudokuCell: {
    flex: 1,
    aspectRatio: 1,
    alignItems: 'center',
    justifyContent: 'center',
    borderWidth: 1,
    borderColor: 'colors.borderDark',
  },
  sudokuCellText: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  sudokuGiven: {
    color: colors.primary,
  },
  sudokuInput: {
    color: colors.warning,
  },
  sudokuSolution: {
    color: colors.success,
  },
  numberPad: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: 6,
    marginBottom: 16,
  },
```

```
  numberBtn: {
    width: '18%',
    aspectRatio: 1,
    backgroundColor: 'colors.borderDark',
    borderRadius: 8,
    alignItems: 'center',
    justifyContent: 'center',
  },
  numberBtnText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: colors.primary,
  },
  clearBtn: {
    width: '18%',
    aspectRatio: 1,
    backgroundColor: 'rgba(255,107,107,0.2)',
    borderRadius: 8,
    alignItems: 'center',
    justifyContent: 'center',
  },
  clearBtnText: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#FF6B6B',
  },
  hintBtn: {
    backgroundColor: 'rgba(255,215,0,0.15)',
    borderRadius: 10,
    padding: 14,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: 'rgba(255,215,0,0.4)',
  },
  hintBtnText: {
    fontSize: 14,
    fontWeight: 'bold',
    color: colors.warning,
  },
  // 推箱子样式
  sokobanHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 12,
  },
  levelName: {
    fontSize: 16,
    fontWeight: 'bold',
    color: colors.warning,
```

```
  },
  sokobanGrid: {
    backgroundColor: colors.backgroundDark,
    borderRadius: 8,
    padding: 8,
    marginBottom: 16,
    borderWidth: 2,
    borderColor: colors.orange,
  },
  sokobanRow: {
    flexDirection: 'row',
    justifyContent: 'center',
  },
  sokobanCell: {
    width: 40,
    height: 40,
    alignItems: 'center',
    justifyContent: 'center',
    borderRadius: 4,
    margin: 1,
  },
  sokobanCellWall: {
    backgroundColor: '#3A3A5C',
  },
  sokobanCellFloor: {
    backgroundColor: colors.background,
  },
  sokobanCellTarget: {
    backgroundColor: 'rgba(255,215,0,0.2)',
  },
  sokobanCellPlayer: {
    backgroundColor: 'colors.borderDark',
  },
  sokobanCellBox: {
    backgroundColor: 'rgba(255,125,0,0.4)',
  },
  sokobanCellBoxOnTarget: {
    backgroundColor: 'rgba(58,227,116,0.4)',
  },
  sokobanCellText: {
    fontSize: 20,
  },
  sokobanControls: {
    alignItems: 'center',
    gap: 4,
    marginBottom: 16,
  },
  sokobanBtnRow: {
    flexDirection: 'row',
    gap: 4,
```

```
  },
  sokobanBtn: {
    width: 55,
    height: 55,
    backgroundColor: 'colors.borderDark',
    borderRadius: 10,
    alignItems: 'center',
    justifyContent: 'center',
    borderWidth: 1,
    borderColor: 'colors.borderDark',
  },
  sokobanBtnText: {
    fontSize: 22,
  },
  answerContainer: {
    backgroundColor: 'rgba(255,215,0,0.08)',
    borderRadius: 10,
    padding: 12,
    marginBottom: 12,
    borderWidth: 1,
    borderColor: 'rgba(255,215,0,0.25)',
  },
  answerTitle: {
    fontSize: 13,
    fontWeight: 'bold',
    color: colors.warning,
    marginBottom: 8,
  },
  answerSteps: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: 8,
    marginBottom: 8,
  },
  stepPending: {
    fontSize: 14,
    color: colors.textSecondary,
  },
  stepDone: {
    fontSize: 14,
    color: colors.success,
  },
  answerButtons: {
    flexDirection: 'row',
    gap: 8,
  },
  smallBtn: {
    flex: 1,
    padding: 8,
    backgroundColor: 'colors.borderDark',
```

```
    borderRadius: 6,
    alignItems: 'center',
  },
  smallBtnText: {
    fontSize: 11,
    color: colors.primary,
    fontWeight: 'bold',
  },
  winOverlay: {
    position: 'absolute',
    top: 0,
    left: 0,
    right: 0,
    bottom: 0,
    backgroundColor: 'rgba(0,0,0,0.7)',
    alignItems: 'center',
    justifyContent: 'center',
    zIndex: 100,
    pointerEvents: 'none',
  },
  winText: {
    fontSize: 32,
    fontWeight: 'bold',
    color: colors.warning,
  },
  submittingText: {
    fontSize: 14,
    color: colors.textSecondary,
    marginTop: 16,
  },
}), [colors]);
  /** 当前显示的屏幕 */
  const [currentScreen, setCurrentScreen] = useState<Screen>('welcome');
  /** 当前选中的游戏类型 */
  const [currentGame, setCurrentGame] = useState<GameType>('sudoku');
  /** 是否游戏胜利 */
  const [gameWon, setGameWon] = useState(false);
  /** 是否正在提交结果 */
  const [isSubmitting, setIsSubmitting] = useState(false);

  /**
   * 处理游戏胜利
   */
  const handleWin = async (level?: number) => {
    console.log('[GameModal] 游戏胜利 - 关卡:', level);
    setGameWon(true);
    setIsSubmitting(true);

    try {
      const result: GameResult = await mockSubmitGameResult(
```

```
        level || 1,
        true,
        monsterType
      );

      if (result.success) {
        console.log('[GameModal] 游戏结果提交成功 - 奖励:', result.rewards);
        setTimeout(() => {
          setGameWon(false);
          setIsSubmitting(false);
          onGameComplete(result.rewards);
        }, 500);
      } else {
        throw new Error(result.message || '游戏提交失败');
      }
    } catch (error) {
      console.error('[GameModal] 游戏提交失败:', error);
      setIsSubmitting(false);
      setGameWon(false);
      Alert.alert(' 提交失败', '游戏结果提交失败，请重试', [
        { text: '重试', onPress: () => handleWin(level) },
        { text: '取消', style: 'cancel' }
      ]);
    }
  };

  /**
   * 显示游戏教程
   */
  const showTutorial = (gameType: GameType) => {
    setCurrentGame(gameType);
    setCurrentScreen('tutorial');
  };

  /**
   * 开始游戏
   */
  const startGame = () => {
    setCurrentScreen('game');
    setGameWon(false);
  };

  /**
   * 返回上一步
   */
  const goBack = () => {
    if (currentScreen === 'game') {
      Alert.alert(' 确认退出', '退出后当前游戏进度将丢失，确定退出吗？ ', [
        { text: '取消', style: 'cancel' },
        { text: '确定', onPress: () => {
```

```
          setCurrentScreen('tutorial');
          setGameWon(false);
        }}
      ]);
    } else {
      setCurrentScreen('welcome');
      setGameWon(false);
    }
  };

  /**
   * 关闭弹窗
   */
  const handleClose = () => {
    if (currentScreen === 'game') {
      Alert.alert(' 确认退出', '退出后当前游戏进度将丢失，确定退出吗？', [
        { text: '取消', style: 'cancel' },
        { text: '确定', onPress: () => {
          setCurrentScreen('welcome');
          setGameWon(false);
          onClose();
        }}
      ]);
    } else {
      setCurrentScreen('welcome');
      setGameWon(false);
      onClose();
    }
  };

  return (
    <Modal
      visible={visible}
      animationType="slide"
      onRequestClose={handleClose}
    >
      <View style={styles.container}>
        {/* 头部栏 */}
        <View style={styles.header}>
          <TouchableOpacity onPress={handleClose} style={styles.closeBtn}>
            <Ionicons name="close" size={28} color={colors.textSecondary} />
          </TouchableOpacity>
          <Text style={styles.title}>游戏恢复</Text>
          <View style={styles.placeholder} />
        </View>

        {/* 欢迎页面 */}
        {currentScreen === 'welcome' && (
          <ScrollView style={styles.welcomeContent} showsVerticalScrollIndicator={false}>
            <View style={styles.welcomeSection}>
```

```
          <Text style={styles.welcomeTitle}> 游戏恢复</Text>
          <Text style={styles.welcomeSubtitle}>完成小游戏，恢复体力和能量！</Text>
        </View>

        {/* 奖励说明 */}
        <View style={styles.rewardsSection}>
          <Text style={styles.sectionTitle}> 游戏奖励</Text>
          <View style={styles.rewardsRow}>
            <View style={styles.rewardBox}>
              <Text style={styles.rewardIcon}></Text>
              <Text style={styles.rewardAmount}>+5</Text>
              <Text style={styles.rewardName}>体力</Text>
            </View>
            <View style={styles.rewardBox}>
              <Text style={styles.rewardIcon}></Text>
              <Text style={styles.rewardAmount}>+3</Text>
              <Text style={styles.rewardName}>能量</Text>
            </View>
          </View>
          <Text style={styles.rewardNote}> 叛逆小怪可获得双倍奖励！</Text>
        </View>

        {/* 游戏选择 */}
        <View style={styles.gamesSection}>
          <Text style={styles.sectionTitle}> 选择游戏</Text>

          <TouchableOpacity
            style={styles.gameCard}
            onPress={() => showTutorial('sudoku')}
          >
            <View style={styles.gameCardIcon}></View>
            <View style={styles.gameCardInfo}>
              <Text style={styles.gameCardTitle}>数独</Text>
              <Text style={styles.gameCardDesc}>
                填写9×9网格，使每行、每列、每个3×3宫格都包含1-9不重复的数字。完成一局即可获得奖励！
              </Text>
            </View>
            <Text style={styles.gameCardArrow}>→</Text>
          </TouchableOpacity>

          <TouchableOpacity
            style={styles.gameCard}
            onPress={() => showTutorial('sokoban')}
          >
            <View style={styles.gameCardIcon}></View>
            <View style={styles.gameCardInfo}>
              <Text style={styles.gameCardTitle}>推箱子</Text>
              <Text style={styles.gameCardDesc}>
                将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
              </Text>
```